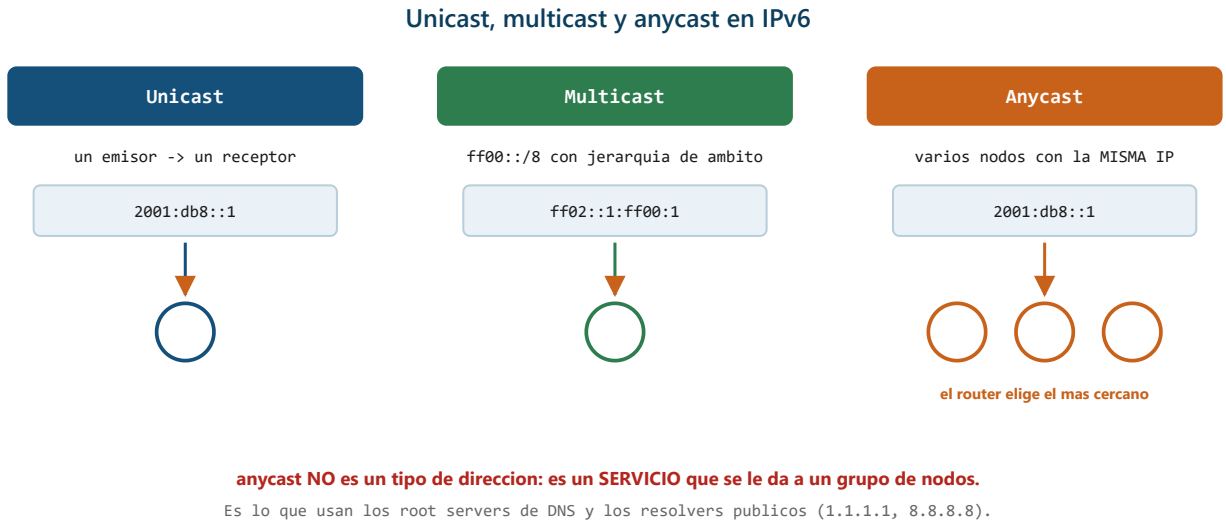
Clase 36 — IPv6

Bloque: D (Capa de Red) · **Semana:** 12 · **Duración estimada:** 120 min **Correlativa:** Clase 35 (Fragmentación, MTU y Path MTU) · **Sigue con:** Clase 37 (ICMP, ping y traceroute + Repaso y Simulacro 4)
Material de referencia: Tanenbaum cap. 5 (§5.6) · RFC 8200, RFC 4443, RFC 6437, RFC 8201

🎯 Al terminar esta clase vas a poder

1. Explicar **por qué** existe IPv6, con los datos de despliegue de 2026 y **diciendo explícitamente que la cifra depende de qué se mide**.
2. Dibujar la **cabecera IPv6** campo por campo y **compararla lado a lado con la IPv4**: qué desapareció y qué es nuevo.
3. Explicar para qué sirve el **Flow Label** (`RFC 6437`) y por qué no alcanza la 5-tupla.
4. Describir la **cadena de cabeceras de extensión** por `Next Header`, y el bug histórico del `Payload Length`.
5. Explicar los **3 modelos de comunicación** y demostrar por qué **anycast no es un tipo de dirección**.
6. Enumerar los **tipos de dirección** y explicar por qué el `/64` **es la unidad de subred**.
7. Describir **ICMPv6** y explicar por qué **no es opcional**.
8. Describir **Neighbor Discovery** y enumerar **los 5 cambios frente a ARP** (el apartado que más puntos da).
9. Explicar **DAD** y por qué no existe en IPv4.
10. Describir los **3 esquemas de transición** con sus ejemplos.



> **Prerequisito del examen:** `2010-12 P5` (8 pts — stacks de referencia, coexistencia y **ND vs. ARP con ≥4 cambios**), `2009-12 P5` (6 pts — DAD y NDP), `2010-07 P5` (8 pts — **3 modelos de**

> **comunicación**, **3 tipos de dirección**, **3 esquemas de transición**), `2007-03 P3` (10 pts — RARP), `2009-07 P5` (ventajas y desventajas de IPv6).

PARTE 1 — TEORÍA

1.1 Por qué IPv6: del "futuro" al presente consumado

El agotamiento es un hecho, no una predicción. IPv4 tiene 2^{32} = 4.294.967.296 direcciones, y no dan. El agotamiento ocurrió por regiones, en este orden:

RIR	Se agotó
APNIC (Asia-Pacífico)	abril de 2011
ARIN (Norteamérica)	septiembre de 2015
AfriNIC (África)	abril de 2017
LACNIC (Latinoamérica)	2020
RIPE NCC (Europa)	**25 de noviembre de 2019**

> **El dato para el aula:** RIPE NCC hizo su **última asignación /22 el 25-nov-2019** a las 15:35 CET. Hoy mantiene una *waiting list* para un único /24 recuperado. IPv4 no "se está agotando": **ya se agotó**. IPv6 tiene 2^{128} direcciones.

El despliegue real en 2026, con el número exacto

> **El 28 de marzo de 2026, el acceso a Google por IPv6 superó el 50 % por primera vez: 50,10 %.** Fuentes: Internet Society Pulse, APNIC (abr-2026), The Register (abr-2026).

Y lo importante es esto: la respuesta a "¿cuánto IPv6 hay?" es "depende de qué midás". Se pregunta:

Quién mide	**Qué** mide	Valor
Google (`stats.google.com/ipv6`)	**usuarios** que acceden a Google por IPv6	**50,10 %** (mar-2026)
APNIC	**cobertura de capacidad** (qué % de redes tiene IPv6)	~**42-43 %**
Cloudflare Radar	**tráfico por bytes**	~**40 %**

Y es desigual: leaders Francia ~73 %, India ~72 %, Arabia Saudita ~65 %; rezagados Italia ~17 %, España ~10 %, Egipto ~4 %, **China < 5 %**.

> **Respuesta modelo para el examen (2012 vs. 2026):** en 2012 el curso decía "IPv6 llega en 10 años". La respuesta correcta hoy es: **"ya pasó la mitad en los usuarios de Google, pero es desigual, China sigue en menos del 5 %, y la cifra depende de qué se mida"**. Una definición de "cuánto IPv6 hay" sin decir qué se midió es una respuesta incompleta.

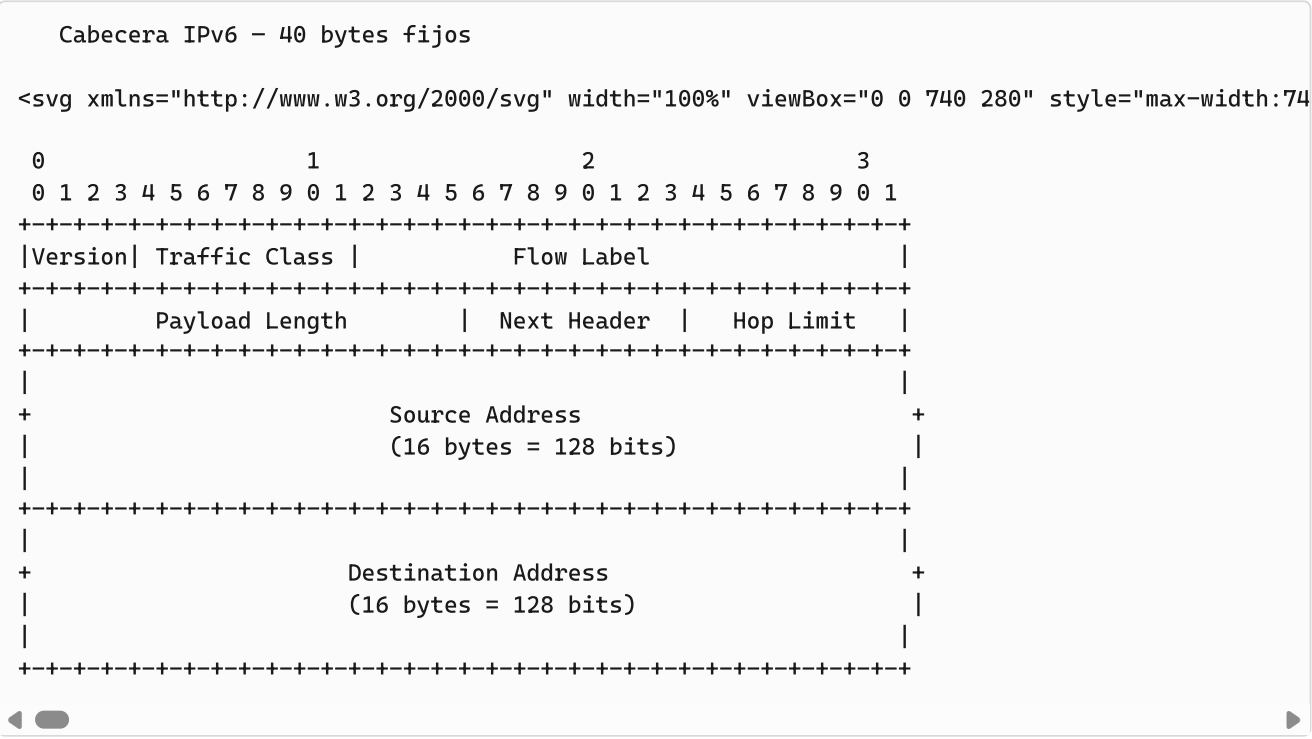
⚠️ Actualización a 2026

- **IPv6 dejó de ser "el futuro".** No hay fecha de cierre de IPv4, y no la habrá pronto.
- **CGNAT (*Carrier-Grade NAT*) es ubicuo en los ISP**: es el nuevo default de la capa 3 de acceso. Rompe conectividad entrante, complica la señalización (SIP/IMS), rompe IPsec y rompe peer-to-peer.
- `RFC 6177` **(2011)** ya no recomienda `/48` por sitio final; los RIR asignan `/48` (a veces `/56`) al **operador**, que reparte.
- **NDP se amplió en 2026:** `RFC 9926` (feb-2026) permite registrar un prefijo en los routers vecinos.
- **Por qué NAT rompe IPv6:** NAT depende de direcciones privadas y reescritura de puertos para multiplexar. IPv6 no **necesita** traducción; peor, NAT rompe la **extensibilidad extremo a extremo** (cabeceras de extensión, movilidad), no es *stateful-friendly* con IPsec, y obliga a los hosts IPv6-only a usar 464XLAT/PLAT para alcanzar contenido IPv4. ⇒ **La meta de IPv6 es eliminar el NAT.**

1.2 La cabecera IPv6

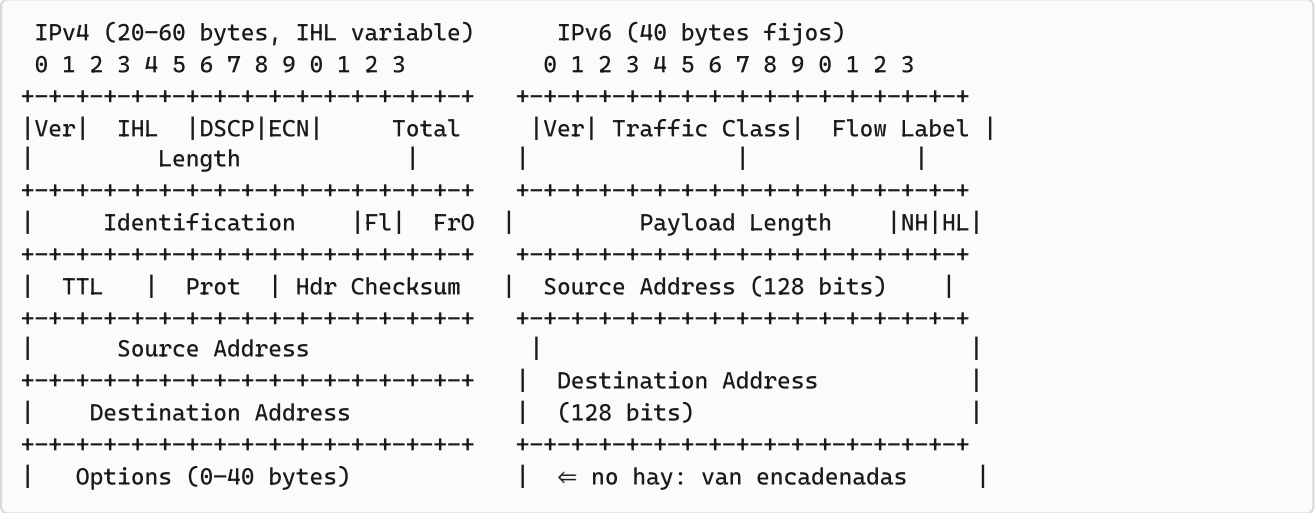
En IPv4 la cabecera mide de 20 a 60 bytes y hay que mirar el `IHL` para saber dónde termina. En IPv6 son **40 bytes fijos**, y lo que sigue son **cabeceras de extensión**.

Cabecera IPv6 — 40 bytes fijos



Campo	Bits	Qué es
Version	4	Siempre **6**.
Traffic Class	8	Equivale a **DSCP + ECN** de IPv4 (6 + 2 bits).
Flow Label	20	**Nuevo.** Identifica un flujo (ver §1.3).
Payload Length	16	Largo del payload en bytes, **sin** las cabeceras de extensión.
Next Header	8	Sustituye al `Protocol` **y** al `IHL`: dice qué va después (TCP=6, UDP=17, ICMPv6=58, o una extensión).
Hop Limit	8	Equivale al `TTL`. Se decrementa en **cada** router.
Source / Destination	128 c/u	

IPv4 contra IPv6, lado a lado



Qué DESAPARECIÓ y por qué:

Desaparecido	Por qué
`Header Checksum`	**Era un desperdicio de CPU** y había que recalcular en **cada** salto porque el TTL cambia. Ahora **el checksum lo pone la capa 4** (TCP, UDP), que verifica **una sola vez extremo a extremo**. Cada router verifica ~0 y el error se detecta igual, pero al final de la tubería.
`IHL`	La cabecera siempre mide 40 bytes. El campo `Next Header` cumple la función doble.
`Fragment Offset`	**IPv6 no fragmenta en los routers.** Los routers solo decrecantan `Hop Limit` y descartan con **Packet Too Big** (tipo 2). Solo el origen fragmenta, con una cabecera de extensión.
`Options`	Se fueron a las cabeceras de extensión. El motivo: en IPv4 una opción **dentro** de la cabecera obligaba a que **todos** los routers la entendieran (o descartaran el paquete). En IPv6 se procesan **solo donde corresponde**.

Qué es NUEVO: el `Flow Label` (20 bits) y las **cabeceras de extensión encadenadas**, que dan **extensibilidad sin cambiar el formato base**.

El argumento profundo: IPv4 optimizó la cabecera para la **velocidad del router** (checksum barato, todo en un lugar). IPv6 la optimizó para la **extensibilidad**, y movió el costo de la verificación al final de la

tubería.

1.3 El Flow Label (`RFC 6437`)

Son **20 bits** que el emisor pone para **identificar un flujo**, y que los routers pueden usar **sin conocer la conexión**.

El problema que resuelve. Un router hace QoS (ECN, remarked DSCP, ECMP por flujo) y necesita saber "este paquete es del mismo flujo que aquél". La forma clásica es la **5-tupla** `(IP origen, puerto origen, IP destino, puerto destino, protocolo)`, que **no siempre se puede construir**:

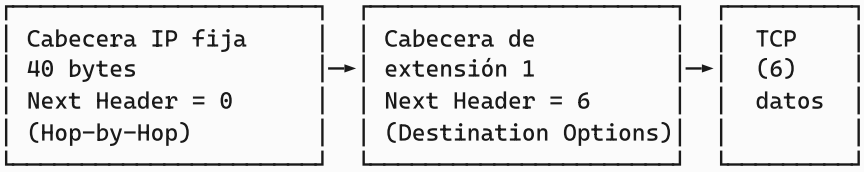
```
1. PAQUETE FRAGMENTADO: solo el PRIMER fragmento tiene la cabecera de
   transporte. Los otros no. Un router que debe hacer ECMP no puede hashear
   un fragmento contra otro sin el Flow Label.

2. PAQUETE CIFRADO: si el transporte va cifrado (IPsec, WireGuard), los
   puertos son ilegibles. El Flow Label está en la cabecera IP, que NO se
   cifra: es visible para un router de medio aunque el payload no lo sea.
```

Reglas de `RFC 6437`: el Flow Label **solo significa algo para un par (origen, destino)**, así que el router identifica el flujo con `(Flow Label, Src, Dst)`; el emisor puede **cambiarlo** cuando cambian los atributos del flujo; los routers que no lo entienden **lo ignoran** (retrocompatible por diseño); y **no es una etiqueta de seguridad**, cualquiera puede poner lo que quiera.

1.4 Cabeceras de extensión y el `Payload Length`

Cuando el `Next Header` no es un protocolo de transporte, **es una cabecera de extensión**. Se encadenan: cada una apunta con su `Next Header` a la siguiente, y la última al protocolo real.



Nombre	`Next Header`	Para qué	¿Quién la procesa?
Hop-by-Hop Options	**0**	Opciones que deben ver **todos** los routers	Todos
Routing	**43**	Encaminamiento explícito. El **tipo 0 fue deprecado** (`RFC 5095`)	Los routers indicados
Fragment	**44**	Solo si el **origen** fragmenta	Solo el destino final
Destination Options	**60**	Opciones para el destino	Solo el destino final
AH	**51**	Integridad y autenticidad	Los que entiendan AH
ESP	**50**	Confidencialidad + integridad + anti-replay	Solo el destino final

> ⚠️ **El bug histórico del `Payload Length`.** En la especificación original (`RFC 2460` , 1998) el `Payload Length` incluía **las cabeceras de extensión**. Eso obligaba a que el nodo que fragmentaba conociera el largo de las cabeceras de extensión de los paquetes que iba a producir. **Era un error de diseño**, y `RFC 2719` (1999) lo corrigió: **el `Payload Length` excluye las cabeceras de extensión**. Si hay 8 bytes de extensión y 1000 de datos, `Payload Length = 1000` . La vigente, `RFC 8200` (2017), ya trae la corrección.

Por qué son la gran victoria de diseño de IPv6: en IPv4, para agregar una opción había que modificar el formato de la cabecera, y **cualquier router que no la entendiera tenía que descartar el paquete**. En IPv6, un router que no entiende una extensión simplemente **la salta** (salvo Hop-by-Hop y Fragment, que son obligatorias). **Las nuevas funciones se pueden agregar sin romper Internet.**

1.5 Los 3 modelos de comunicación

Este es el `2010-07 P5 a)` .

1) Unicast — un emisor, un receptor

El modelo clásico: cada dirección identifica **exactamente un** nodo.

```
H1 (2001:db8::1)  ──────────────►  S (2001:db8::2)
     emisor                          receptor
```

La dirección de **destino** identifica al destino, la de **origen** al emisor, y ambos van en la cabecera. Es el 99 % del tráfico.

2) Multicast — un emisor, muchos receptores

Prefijo `ff00::/8` : el primer byte de la dirección es `ff` .

```
|1111 1111| 4 bits de ámbito (scope) | 4 bits de flag | 112 bits de ID |
└ ff ─────┘                          └── flags ──────────┴── del grupo ──┘
```

Ámbito	Prefijo	Significado
0	`ff00::/8`	Reservado
1	`ff01::/16`	**Node-local** (solo en el propio host)
2	**`ff02::/16`**	**Link-local**: solo en el mismo enlace
5	`ff05::/16`	Site-local (dentro de un sitio)
8	`ff08::/16`	Organization-local
14	`ff0e::/16`	**Global**
15	`ff0f::/16`	Reservado

⚠️ **`site-local`** (**`fec0::/10`**) **está DEPRECADO**: se reemplaza por **unique local** (`fc00::/7`).

La jerarquía: un router que recibe tráfico para `ff02::/16` **no lo reenvía fuera del enlace**. Uno que recibe para `ff0e::/16` lo reenvía por Internet.

Ejemplo real: IPv6 usa multicast siempre, aunque el usuario no lo sepa (`ff02::1` = todos los nodos del enlace, `ff02::2` = todos los routers del enlace, `ff01::1` = el propio nodo).

3) Anycast — muchos emisores, muchos receptores

> ⚠️ **Punto clave, y es un error clásico de examen: el anycast NO es un tipo de dirección.** Es un **servicio** que se le asigna a un grupo de nodos que tienen **la misma dirección unicast**, y el router encamina al **más cercano**.

```
⇐ TODOS tienen la MISMA dirección: 2001:db8::1
⇐ NO hay prefijo especial, NO hay multicast, NO hay bit que lo marque

┌──────────┐   ┌──────────┐   ┌──────────┐
│ Nodo A   │   │ Nodo B   │   │ Nodo C   │
│ 2001:db8 │   │ 2001:db8 │   │ 2001:db8 │
│   ::1    │   │   ::1    │   │   ::1    │
└──────┬───┘   └──────┬───┘   └──────┬───┘
       └──────────────┼──────────────┘
                      ▼
          ┌──────────────────┐
          │ "encaminá a      │
          │  2001:db8::1 al  │
          │  más cercano"    │
          └──────────────────┘
```

¿Por qué no es un tipo de dirección?

1. **No hay prefijo ni bit que lo marque.** Es una dirección unicast normal, asignada a varios nodos a la vez. La tabla de ruteo es la de siempre.
2. **La decisión es del router, no del paquete.** El paquete **no sabe** que hay más de uno.

3. **El receptor no se identifica.** El emisor no puede saber a cuál de los tres le llegó.

¿Cómo es posible? Porque en ruteo **la dirección destino identifica un destino lógico, no un host físico**: con BGP, un AS anuncia el prefijo desde **varios puntos** del mundo y el cliente elige el más cercano.

Dónde se usa: los **13 root servers de DNS** (todos con `198.41.0.4` y `2001:503:ba3e::2:30`), los **resolvers públicos** (`8.8.8.8`, `1.1.1.1`), y **los CDNs**.

La comparación clave (multicast vs. anycast):

	Multicast	Anycast
¿Tipo de dirección?	**Sí**: `ff00::/8`	**NO** — servicio sobre unicast
¿A quién llega?	A **todos** del grupo	Al **más cercano** (uno solo)
¿Cómo lo decide?	El **paquete** (dirección)	El **router** (encaminamiento)
Configuración	Requiere **IGMP/MLD** y ruteo multicast	**Ninguna**: es ruteo normal
Ejemplo	`ff02::1`	`8.8.8.8`

⚠️ Errores típicos

- **Decir que anycast es un tipo de dirección.** No existe prefijo anycast.
- **Confundir los bits de ámbito con los de flag** en la dirección multicast.
- **Decir que site-local es válido.** Está deprecado; se usa `unique local`.

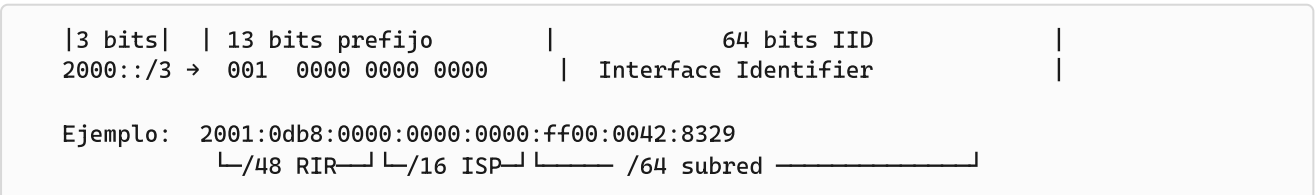
1.6 Los tipos de dirección IPv6

El enunciado de `2010-07 P5 a)` pide tres; la realidad son cuatro. Los tres que el curso emphasizes:

Tipo	Prefijo	Alcance	¿Cómo se obtiene?
Unicast global	`2000::/3`	Internet entero	Asignado por un RIR a través de un ISP
Link-local	`fe80::/10`	**Solo en el mismo enlace**	**Se genera sola** en cada interfaz
Multicast	`ff00::/8`	Según el ámbito	Configurado por la aplicación
Unique local	`fc00::/7`	Prefijo único de la organización	Análogo a las privadas IPv4

La link-local `fe80::/10` deserves atención: se genera automáticamente en cada interfaz sin configurar nada; **no se enruta** (un router jamás reenvía un paquete link-local fuera del enlace, por diseño); y es la dirección que la máquina usa **antes** de tener dirección global y para hablar con sus vecinos.

El formato, de memoria:



El `/64` como unidad de subred, y por qué

Regla que hay que saber de memoria: en IPv6 el `/64` es la unidad de subred. No el `/48`, no el `/56`: **el `/64`**.

¿Por qué 64 bits de IID? Porque la generación automática (*SLAAC*) necesita **64 bits** para el identificador de interfaz:

```
1. EUI-64 (derivado de la MAC):
     MAC:  00:1B:44:11:3A:B7
     Se invierte el primer byte y se inserta FF:FE en el medio:
     IID:  02:1B:44:FF:FE:11:3A:B7
     ⚠️ Desventaja: preserva la MAC ⇒ la dirección ES un identificador
        único persistente ⇒ Privacy Extensions lo resolvió.

2. Privacy Extensions (RFC 4941): el default actual.
     IID ALEATORIO de 64 bits, que cambia cada cierto tiempo (días u horas),
     para que un observador externo NO pueda correlacionar la actividad a
     lo largo del tiempo. Default en Windows, macOS y Linux.
```

Los prefijos reservados y de uso, de memoria:

```
2000::/3   Unicast global          ← el 99 % de las direcciones
fc00::/7   Unique Local            ← como las privadas IPv4
fe80::/10  Link-Local              ← se genera sola
ff00::/8   Multicast               ← los grupos
(el resto: reservado para uso futuro)
```

1.7 ICMPv6 (`RFC 4443`)

> ⚠️ **ICMPv6 NO es opcional: es parte esencial de IPv6.** Un host IPv6 sin ICMPv6 **no funciona**: no resuelve direcciones de capa 2, no se configura solo, no sabe si un paquete es demasiado grande.

La razón profunda: IPv6 absorbió lo que en IPv4 eran **protocolos independientes** (ARP, RARP, ICMP) y lo **integró en un solo**, reduciendo el número de protocolos de la pila.

El formato del mensaje es el de ICMPv4 (tipo, código, checksum, cuerpo), con un campo de 32 bits obligatorio que identifica la cabecera IPv6 que generó el error.

Tipo	Nombre	Para qué
1	Destination Unreachable	Análogo al tipo 3 de IPv4
2	**Packet Too Big**	Lo emite el router cuando el paquete no entra. **Lleva el MTU**, y el origen **tiene que fragmentar**
3	Time Exceeded	Análogo al tipo 11. Lo usa `traceroute`
4	Parameter Problem	Análogo al tipo 12
128	**Echo Request**	`ping6`
129	**Echo Reply**	Respuesta de `ping6`
133	**Router Solicitation**	El host pide routers del enlace
134	**Router Advertisement**	El router se anuncia y manda prefijo y MTU
135	**Neighbor Solicitation**	"Who has X?" — el **ARP request** de IPv6
136	**Neighbor Advertisement**	"This is me" — el **ARP reply** de IPv6
137	**Redirect**	El router sugiere un mejor camino

Los cambios de numeración que hay que tener claros:

```
ICMPv4      ICMPv6        Cambio
──────      ──────        ──────────────────────────────────
Tipo 0      Tipo 129      Echo Reply:   0 → 129
Tipo 8      Tipo 128      Echo Request: 8 → 128
Tipo 3/4    Tipo 2        "Frag Needed" → "Packet Too Big"
Tipo 11     Tipo 3        Time Exceeded: 11 → 3
Tipo 12     Tipo 4        Parameter Problem: 12 → 4
(n/a)       Tipo 133–137  Neighbor Discovery (nuevo)
```

La diferencia crítica del tipo 2 frente al 3/4 de IPv4:

```
IPv4, tipo 3 código 4:  el router DICE "necesitás 1492" y el emisor
                        AJUSTA su MSS. El paquete NO se fragmenta más.

IPv6, tipo 2:           el router DICE "no entra, te paso 1280" y el
                        ORIGEN tiene que FRAGMENTAR él mismo (porque los
                        routers IPv6 NO pueden fragmentar). Después el
                        siguiente paquete va más chico, y así.
```

1.8 Neighbor Discovery vs. ARP — el apartado que más puntos da

Este es el `2010-12 P5 b)`, que pide "**no menos de 4 cambios significativos**". La solución oficial lista 5.

ARP (`RFC 826`) responde una pregunta: "**¿qué MAC corresponde a esta IP?**", usando **broadcast** de capa 2, y es un **protocolo independiente** definido aparte de IP.

Neighbor Discovery (ND) es parte de **ICMPv6** y resuelve **tres** preguntas:

```
ARP resuelve UNA cosa:
    1. ¿Qué MAC corresponde a esta IP?          (resolución de dirección)

ND resuelve TRES cosas:
    1. ¿Qué MAC corresponde a esta IP?          (resolución de dirección)
    2. ¿Qué prefijo y qué parámetros hay acá?   (autoconfiguración SLAAC)
    3. ¿Hay alguien más con MI dirección?       (detección de duplicados DAD)
```

Los 5 cambios significativos

(1) ND es parte de ICMPv6; ARP es un protocolo independiente.

```
IPv4:  App — TCP — IP — ICMP     ARP vive APARTE, en un ethertype distinto
                                  (0x0806), con su propio formato.

IPv6:  App — TCP — IPv6 — ICMPv6
                              └── ND (tipos 133-137) vive AQUÍ ADENTRO.
                                  No existe "ARP" en IPv6.

Por qué importa: una sola pila de protocolos, menos superficie de ataque,
menos código en el kernel, y **es imposible desactivar ND sin romper IPv6**.
```

(2) ND usa MULTICAST en lugar de BROADCAST.

```
ARP request:  FF:FF:FF:FF:FF:FF  (broadcast). TODOS lo reciben y lo procesan.

ND request:   dirección multicast SOLICITED-NODE,  ff02::1:ffxx:xxxx
              Se toman los últimos 24 bits de la IPv6 buscada y se anteponen
              ff02:0:0:0:0:1:ff00:

              Ejemplo: buscar 2001:db8::1234:5678:9abc
                últimos 24 bits:              34:5678:9abc
                solicitation:      ff02::1:ff00:34:5678:9abc

¿Por qué es mejor? Porque los routers NO reenvían ese multicast, así que el
scope NO sale del enlace. Con el broadcast de ARP, un router con proxy ARP
responde y el broadcast inunda cada segmento. El multicast es más eficiente
y más seguro: solo los nodos INTERESADOS responden.
```

(3) Los mapeos son link-layer ↔ IPv6 en vez de link-layer ↔ IPv4.

Cambio directo: ND mapea **MAC ↔ IPv6**, no IPv4. En la misma red pueden coexistir ARP (para IPv4) y ND (para IPv6), con cachés separadas.

(4) ND combina la resolución con la AUTOCONFIGURACIÓN y la DETECCIÓN DE DUPLICADOS.

Es el cambio conceptual más grande. En IPv4, ARP **solo resuelve MACs**: para tener una IP hay que usar DHCP o configurar a mano. En IPv6, ND hace las tres cosas, lo que permite **IPv6 *zero-touch***: el host se enchufa y **se configura solo**.

(5) Los nodos pueden usar mensajes NS para propagar cambios en su configuración.

```
En IPv4, si un host cambia su IP (failover de cluster, cambio de VLAN), los
demás se quedan con la caché ARP vieja hasta que expira. ARP no tiene un
"anuncio" de la IP nueva.

En IPv6, ND permite:
  - Anunciar la dirección nueva proactivamente (NA no solicitado) para que
    los vecinos actualicen su Neighbor Cache.
  - Detectar un nodo que se va (responde al NS con NA con Override = 1).
  - Detectar duplicados.
```

Tabla comparativa completa (esto es lo que hay que saber de memoria):

Aspecto	**ARP (IPv4)**	**Neighbor Discovery (IPv6)**
¿Qué protocolo es?	Independiente, `RFC 826`	**Parte de ICMPv6** (`RFC 4443`)
Corre sobre	El enlace (Ethernet)	**IPv6**
Identificador	Ethertype `0x0806`	Tipo ICMPv6 **135** (NS) / **136** (NA)
Solicitud	**Broadcast** (`FF:FF:...`)	**Multicast** a `ff02::1:ffxx:xxxx`
Respuesta	Unicast	Unicast (o multicast con Override)
Mapeo	MAC ↔ **IPv4**	MAC ↔ **IPv6**
Funciones	**1**: resolución	**3**: resolución + **autoconfig** + **DAD**
Detección de duplicados	**No**	**Sí** (DAD)
Autoconfiguración	**No** (DHCP o manual)	**Sí** (SLAAC vía RA)
Proxy ARP	**Sí**, muy usado	**No** (el modelo es punto a punto)
Mensajes	2 (request, reply)	**5** (RS, RA, NS, NA, Redirect)
Seguridad	Cero: ARP spoofing trivial	Similar; **secg** lo extiende

1.9 DAD — Duplicate Address Detection

El problema que resuelve. En IPv4, si dos equipos tienen la misma IP, el resultado es errático (cada uno contesta por su tabla ARP) y **nadie te avisa**.

En IPv6 esto sería inaceptable, porque **las direcciones se generan automáticamente**: es muy probable que dos hosts generen la misma. **Por eso existe DAD.**

```
Un host H quiere usar la dirección X (link-local o SLAAC).

Paso 1:  H pone X en estado "Tentative": todavía no la usa para nada.
         Arranca el RetransTimer en 1 segundo.

Paso 2:  H manda un Neighbor Solicitation (NS) para X, a la dirección
         solicited-node de X: "¿hay alguien con X?"

Paso 3:  H espera 1 segundo.

Paso 4:  ¿Recibió algún Neighbor Advertisement?
           - SÍ → alguien tiene esa dirección  ⇐  H NO USA X, elige otra.
           - NO → nadie la tiene               ⇐  X pasa a "Preferred" y se usa.
```

¿Por qué DAD no existe en IPv4? Tres razones:

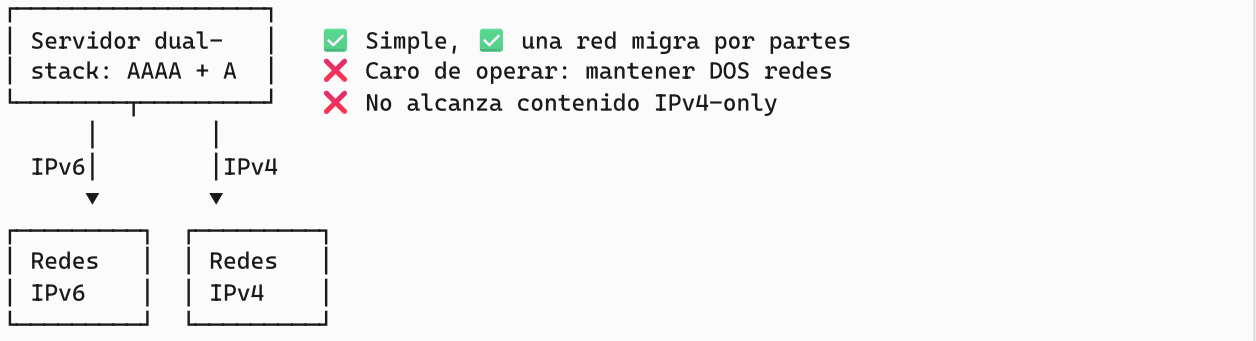
1. **En IPv4 la dirección se configura manualmente o por DHCP.** Dos hosts con la misma IP es un error humano raro. En IPv6 con **SLAAC** la dirección se genera automáticamente, así que la colisión es **estructuralmente más frecuente**.
2. **Dos NICs que comparten la MAC** (fallo de hardware, VM mal configurada) generan **exactamente la misma dirección**. **DAD detecta eso.**
3. **DAD necesita multicast** (NS/NA) para funcionar, y en IPv4 el ARP request es un **broadcast** que no sirve así (el ARP resuelve *después* de que la IP ya está en uso). **DAD inherentemente necesita el modelo de IPv6.**

1.10 Los 3 esquemas de transición

Este es el `2010-07 P5 b)`.

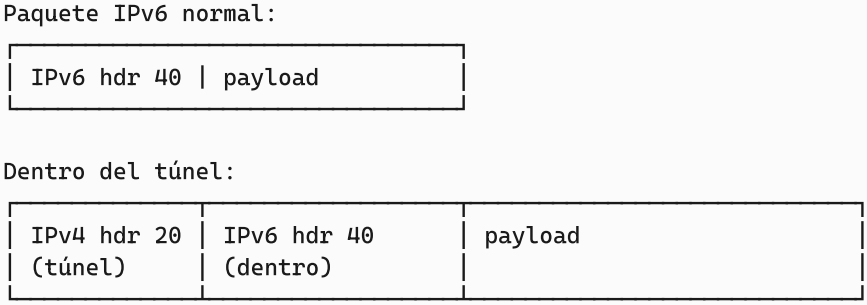
(a) Dual-stack — IPv4 e IPv6 en paralelo

Ambos protocolos en el mismo host y en la misma red. El tráfico elige versión según lo que se pueda resolver.



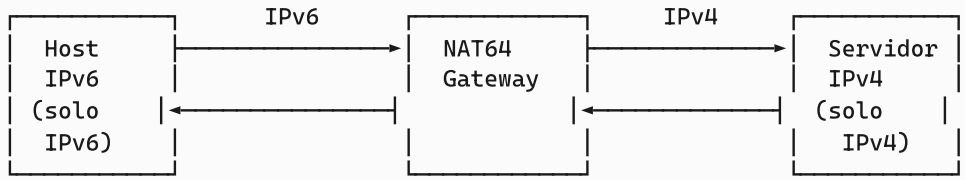
Ejemplo: hoy casi todo servidor web tiene **AAAA y A**. El navegador, con **Happy Eyeballs** (`RFC 8305`), los prueba en paralelo.

(b) Tunnelling — encapsular IPv6 dentro de IPv4

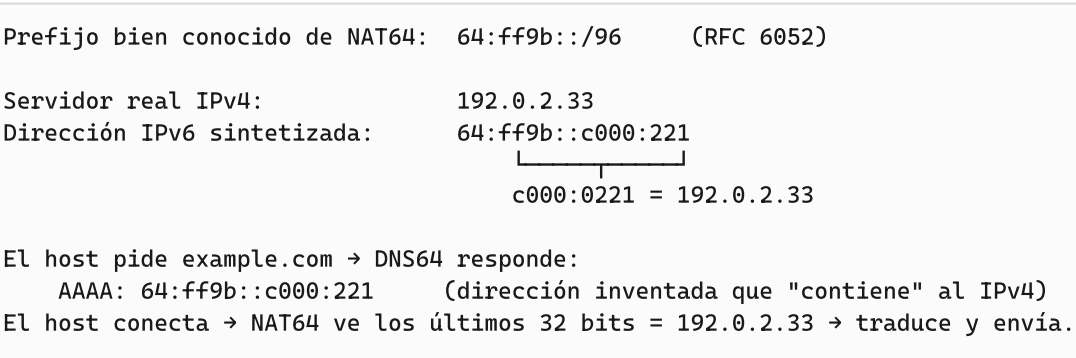


Túnel	Qué es	Para qué
Teredo	IPv6 dentro de IPv4 sobre **UDP/3544**	**NAT traversal**: dar IPv6 a hosts detrás de NAT IPv4. Asigna la IPv6 desde un servidor Teredo (`2001::/32`)
6to4	IPv6 dentro de IPv4 (protocolo 41)	Conectar **sitios IPv6 a través de la IPv4 Internet**, sin servidor. El prefijo IPv6 **embebe** el IPv4 público del sitio (`2002::/16`)
ISATAP	IPv6 dentro de IPv4 en el **mismo enlace**	Traducir IPv4↔IPv6 **dentro de una LAN**. IPv6 derivada del IPv4 del host

(c) Traducción — NAT64 / DNS64



El problema de DNS64: para que el host IPv6-only convierta un nombre a IPv6, el DNS64 devuelve una dirección IPv6 **sintetizada** que contiene la IPv4 real:



Protocolo	RFC	Año	Qué
NAT64	**`RFC 6146`**	2011	Traducción IPv6↔IPv4
DNS64	**`RFC 6147`**	2011	Síntesis de registros AAAA
Prefijo `64:ff9b::/96`	**`RFC 6052`**	2010	El prefijo bien conocido
`ipv4only.arpa`	`RFC 8880`	2020	Preguntar si un dominio tiene IPv4
SIIT	`RFC 6145` → **`RFC 7915`**	2011→2016	Traducción *stateless* (obsoleto)
DS-Lite	**`RFC 6333`**	2011	Dual-stack *stateless* ligero
464XLAT	**sigue en borrador**	—	Traducción en el **host**, no en la red

464XLAT en dos palabras: el **PLAT** (en el router de casa) traduce la salida, y el **CLAT** (en el dispositivo) traduce las apps que no soportan IPv6. Sirve para que un dispositivo **IPv6-only** alcance contenido **IPv4-only**. **La transición no cerró**, y ese es el dato de 2026.

	Dual-stack	Tunnelling	Traducción
Qué hace	Ambos en paralelo	Encapsula IPv6 en IPv4	Traduce IPv6↔IPv4
Ejemplo	Servidor con AAAA + A	Teredo, 6to4, ISATAP	NAT64/DNS64, 464XLAT
Alcanza IPv4-only	❌	✅	✅
IPv6 nativo	✅	❌	❌
Rompe el extremo a extremo	No	Sí (túnel)	**Sí**

1.11 La fragmentación en IPv6: el router NUNCA fragmenta

```
IPv4:  el ORIGEN manda un paquete grande
         ↓  el router lo FRAGMENTA si no cabe (con DF = 0)
       el destino REENSAMBLA

IPv6:  el ORIGEN manda un paquete grande
         ↓  el router NO PUEDE fragmentar
         ↓  lo DESCARTA y manda ICMPv6 "Packet Too Big" (tipo 2)
         ↓  el ORIGEN, avisado, FRAGMENTA con la cabecera de extensión
         ↓  y si tampoco entra, repite con un tamaño menor
```

¿Por qué? Porque la fragmentación en los routers es **el peor problema de IPv4** (overhead, solapamiento como agujero de seguridad, evasión de filtros, pérdida de un fragmento = pérdida del datagrama). **IPv6 lo elimina de raíz.**

Consecuencias positivas: routers **más simples y rápidos** (sin estado de reensamblaje ni offsets); el **ataque de fragmentación en capas desaparece**; el **PMTUD es obligatorio y bien definido**; y el **solapamiento se prohíbe** por diseño (`RFC 5722`).

	IPv4	IPv6
¿Quién fragmenta?	El **router** (si `DF = 0`)	**Solo el origen**
¿Cómo se señala?	Flags + Fragment Offset	**Cabecera de extensión Fragment** (44)
Si no cabe	Con `DF=1` descarta + ICMP 3/4; con `DF=0` **fragmenta**	Siempre descarta + **ICMPv6 tipo 2**
Solapamiento	Permitido en la práctica (agujero de seguridad)	**Prohibido** (`RFC 5722`)
Fragmentos atómicos	No existen	Sí, por compatibilidad (`RFC 6946`)
MTU mínimo	68 absoluto, 576 práctico	**1280** obligatorio

> ⚠️ **IPv6 fragmenta de otra manera:** en IPv4, si un paquete pasa por 3 routers con MTUs decrecientes, se fragmenta **en cada salto**, generando una cascada de fragmentos. En IPv6 el origen prueba, recibe el aviso del primer router que no pasa, y produce **todos los fragmentos de una vez**.

1.12 Actualización a 2026 — IPv6

Lo que sigue igual, de memoria, sin dudar:

- Dirección de **128 bits**, notación hex y compresión con `::`.
- **Cabecera fija de 40 bytes**: Version, Traffic Class, **Flow Label**, Payload Length, Next Header, Hop Limit, Src, Dst.
- **Cabeceras de extensión** encadenadas por `Next Header`. El **Payload Length excluye** las cabeceras de extensión.
- **Ámbitos**: unicast global `2000::/3`, **link-local** `fe80::/10`, unique local `fc00::/7`, multicast `ff00::/8`. **Site-local está deprecado.**
- **`/64` es la unidad de subred** estándar.
- **ICMPv6 (`RFC 4443`)**: ND (RS/RA/NS/NA, Redirect), **Packet Too Big (tipo 2)**, Destination Unreachable (1), Time Exceeded (3), Parameter Problem (4).
- **ND es "ARP para IPv6"** pero en **multicast sobre ICMPv6**, e incluye **autoconfiguración** y **DAD**.
- **Un router IPv6 NO fragmenta.** El **origen** fragmenta.
- **MTU mínimo absoluto de IPv6 = 1280** (`RFC 8200 §5`, verificado textualmente).

Lo que cambió respecto de 2012:

- **IPv6 dejó de ser "el futuro"**: el número del 28-mar-2026 y sus matices (§1.1).
- **El agotamiento de IPv4 es un hecho consumado** (RIPE, 25-nov-2019).
- **CGNAT es ubicuo en los ISP.**
- **NAT64/DNS64 (`RFC 6146` / `6147`) maduraron**; prefijo `64:ff9b::/96` = `RFC 6052`; `RFC 8880` (`ipv4only.arpa`).
- **SIIT `RFC 6145`** → **obsoleto por `RFC 7915`** (2016).

- **464XLAT sigue en borrador** — la transición no cerró.
- **DS-Lite =** `RFC 6333`.
- **NDP se amplió:** `RFC 9926` (feb-2026) registra prefijos entre routers vecinos.
- `RFC 6177` ya no recomienda `/48` por sitio final.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Escribir una cabecera IPv6 a partir de datos

Datos. Versión 6 · Traffic Class `0x93` (DSCP 0b100100 = 36 = AF31, ECN = 11 = CE) · Flow Label `0x0ABCDE` · Payload Length **1440** · Next Header **6** (TCP) · Hop Limit **64** · Src `2001:0DB8:0001:0000:0000:0000:0000:0001` · Dst `2001:0DB8:0002:0000:0000:0000:0000:0002`.

a) Campo por campo.

Campo	Valor	En hexadecimal	Bytes
Version	6 (4 bits) = `0110`	—	byte 0 (nibble alto)
Traffic Class	`0x93`	`93`	byte 0 (nibble bajo) + byte 1
Flow Label	`0x0ABCDE` (20 bits)	`0A BC DE`	bytes 2, 3, 4
Payload Length	1440	**`05 A0`**	bytes 5, 6
Next Header	6	`06`	byte 7
Hop Limit	64	`40`	byte 8
Source Address	`2001:0DB8:0001::1`	ver abajo	bytes 9–24
Destination Address	`2001:0DB8:0002::2`	ver abajo	bytes 25–40

b) Las direcciones, byte a byte. Cada grupo de 4 hex es un `uint16`:

```
2001:0DB8:0001:0000:0000:0000:0000:0001
↓    ↓    ↓    ↓    ↓    ↓    ↓    ↓
20 01  0D B8  00 01  00 00  00 00  00 00  00 00  00 01
```

c) Los 40 bytes completos, en hex y en orden:

```
Bytes  0–1:  69 3A          (Version 6, TC 0x93)
Bytes  2–4:  0A BC DE       (Flow Label 0x0ABCDE)
Bytes  5–6:  05 A0          (Payload Length = 1440)
Byte   7:    06             (Next Header = TCP)
Byte   8:    40             (Hop Limit = 64)
Bytes  9–24: 20 01 0D B8 00 01 00 00 00 00 00 00 00 00 00 01
Bytes 25–40: 20 01 0D B8 00 02 00 00 00 00 00 00 00 00 00 02
```

d) Verificación del `Payload Length`.

```
0x05A0 = 5 × 256 + 10 × 16 + 0 = 1280 + 160 = 1440  ✓
```

Interpretación. La cabecera ocupa **40 bytes fijos**. El `Payload Length` de 1440 **no incluye** esos 40: es el contenido. El **datagrama completo mide 1480 bytes** (40 + 1440). Si pasa por un enlace de MTU 1400, el router **descartará** y mandará **ICMPv6 tipo 2**.

e) ¿Y el MSS? En Ethernet (MTU 1500) el MSS TCP sería `1500 − 40 − 20 = 1420`, no 1460 como en IPv4. **Esa es la penalización del overhead:** 20 bytes más de cabecera IP.

2.2 Ejercicio 2 — Multicast vs. anycast: por qué anycast no es un tipo de dirección

a) Las diferencias.

	Multicast	Anycast
¿Tipo de dirección?	**Sí** (prefijo `ff00::/8`)	**No.** Servicio sobre unicast
¿Prefijo que lo marque?	**Sí**: `ff`	**No**
¿Lo sabe el emisor?	No	No
¿Lo sabe el router?	Por el prefijo `ff`	Por la **tabla de ruteo** (varios puntos anuncian el prefijo)
¿A cuántos destinatarios?	**Todos** del grupo	**Solo al más cercano**
Configuración	Ruteo multicast (IGMP/MLD)	**Ninguna**

b) El argumento formal. Un **tipo de dirección** es un **prefijo reservado** que le dice a la red "esto es especial": `ff00::/8` es multicast y **cualquier router lo reconoce por el primer byte**; `fe80::/10` es link-local y los primeros 10 bits lo dicen.

```
IPv6 multicast:   |1111 1111| ...   ⇐ el primer byte ES ff, se ve
IPv6 link-local:  1111 1110 10..    ⇐ los primeros 10 bits lo dicen
IPv6 unicast:     001 ...            ⇐ 2000::/3

IPv6 anycast:     001 ...            ⇐ EXACTAMENTE IGUAL que un unicast
                                     ⇐ No hay nada que lo distinga
```

El anycast no tiene prefijo reservado. La unicidad la resuelve el ROUTEO, no la numeración. El router ve que el prefijo está anunciado desde varios AS y elige el más cercano. El **paquete no sabe** que hay más de un destino: son **varios nodos que comparten la misma dirección**.

c) El ejemplo: los root servers de DNS.

```
Los 13 root servers de DNS tienen TODOS la misma dirección IPv4: 198.41.0.4
y la misma IPv6: 2001:503:ba3e::2:30

Cada uno está en un lugar distinto del mundo. Un ISP anuncia el prefijo
198.41.0.0/24 desde CUALQUIERA de sus puntos de presencia.

⇐ Tu consulta DNS va al root MÁS CERCANO, no al "primero" de la lista.
⇐ "Any" = cualquiera de los 13; "cast" = difundir.
```

d) Consecuencia práctica: anycast es un servicio de red, no una propiedad de la dirección.

```
✅ Puedo poner la MISMA dirección en 5 servidores míos y funciona: es anycast.
❌ NO puedo "convertir" una dirección existente en anycast sin anunciarla en
   BGP desde 5 puntos distintos. Eso es una decisión de RUTEO.
⚠️ Si el anycast está mal configurado (vos y tu vecino anuncian el prefijo),
   algunos routers pueden enviar tráfico al lugar equivocado. Eso es un
   problema de BGP, no de IPv6.
```

2.3 Ejercicio 3 — ND vs. ARP en tabla comparativa completa

Este es el `2010-12 P5 b)`, con la respuesta lista para el examen.

a) El reemplazo de ARP: Neighbor Discovery (ND). Un **conjunto de 5 mensajes de ICMPv6**: RS (133), RA (134), NS (135), NA (136) y Redirect (137).

b) Los 5 cambios significativos (los de la solución oficial):

#	Cambio	Por qué importa
1	**ND es parte de ICMPv6, en su totalidad; se transporta sobre IPv6**	ARP era independiente, sobre el enlace. ND vive dentro de ICMPv6. Menos protocolos, y **no se puede desactivar sin romper IPv6**.
2	**Se utiliza multicast para Neighbor Solicitation en vez de broadcast**	Va a `ff02::1:ffxx:xxxx` (multicast link-local), no a `FF:FF:FF:FF:FF:FF`. Los routers **no reenvían** ese multicast: **no hay inundo de broadcast**.
3	**Los mapeos son de direcciones link-layer a IPv6 en vez de link-layer a IPv4**	Cambio directo: ND resuelve MAC ↔ IPv6.
4	**ND combina la resolución de direcciones con autoconfiguración y detección de duplicados**	ARP solo resuelve MACs. ND además **configura el host** (SLAAC) y **detecta colisiones** (DAD). Habilita el IPv6 *zero-touch*.
5	**Los nodos pueden usar mensajes NS para propagar cambios en su configuración**	Si un host cambia de dirección, manda un **NA no solicitado** para actualizar la caché de los vecinos. Con ARP, los demás quedan con la caché vieja hasta que expira.

c) La tabla comparativa completa está en §1.8 (última tabla), con 12 filas: protocolo, capa, identificador, solicitud, respuesta, mapeo, funciones, DAD, autoconfiguración, proxy ARP, cantidad de mensajes y seguridad.

2.4 Ejercicio 4 — Los stacks de referencia IPv4 e IPv6

Este es el `2010-12 P5 a)`.

```
Stack IPv4 (referencia)                Stack IPv6 (referencia)
──────────────────────                 ──────────────────────
Aplicación:                            Aplicación:
  HTTP, FTP, SMTP, DNS, SNMP,            HTTP, FTP, SMTP, DNS,
  Telnet                                 SNMP, Telnet

Transporte:                            Transporte:
  TCP (6), UDP (17)                      TCP (6), UDP (17)

Red:                                   Red:
  IPv4 + ICMPv4                          IPv6 + ICMPv6
  (puede fragmentar en routers)          (NUNCA fragmenta; ICMPv6 obligatorio)

Enlace:                                Enlace:
  Ethernet (802.3), PPP, HDLC,           Ethernet (802.3), PPP,
  Frame Relay, ATM, 802.11               ATM, 802.11

Física:                                Física:
  cobre, fibra, radio, satélite          cobre, fibra, radio, satélite
```

Protocolos de capa SUPERIOR con los que coexisten (≥2):

Protocolo	Puerto	¿Coexiste con IPv6?
TCP	6	**Sí**, es el mismo. Solo cambia la cabecera IP que lo lleva
UDP	17	**Sí**, igual
DNS	53	**Sí**, con los registros **AAAA** además de **A**
HTTP	80	**Sí**, el mismo protocolo; cambia solo el transporte debajo
SNMP	161	**Sí**, con *discovery* IPv6
DHCP	67/68	En IPv6 se usa **DHCPv6** (546/547), que es un protocolo **distinto**

Protocolos de capa INFERIOR con los que coexisten (≥2):

Protocolo	Por qué coexiste
Ethernet (802.3)	Los dos tipos de paquete viajan por el mismo cable; se distinguen por el **Ethertype**: `0x0800` = IPv4, `0x86DD` = **IPv6**
PPP	Un enlace PPP negocia **IPCP** (IPv4) y **IPV6CP** (IPv6) en la misma conexión
802.11 (Wi-Fi)	Un access point transmite IPv4 e IPv6 indistintamente
ATM / MPLS	Los túneles llevan IPv4 o IPv6 con la misma encapsulación

d) El punto clave: la capa 2 no cambia. Ethernet y Wi-Fi son **agnósticos** del contenido: solo miran el Ethertype. En IPv6 los dispositivos de capa 2 (switches, bridges) **no se modifican**; solo cambian los de capa 3 y 4. Esto es muy distinto de la era de NAT con IPv4, donde el CGN es un problema de capa 3 que **no existe en IPv6 puro**.

e) Por qué la coexistencia es obligatoria: (1) no todos los dispositivos soportan IPv6, y la transición tarda años; (2) la Internet actual es IPv4, así que un host IPv6-only necesita traducir o tunelizar; (3) **es la única migración posible**: no se cambia Internet de un día para otro, se hace dual-stack primero, túnel o transición después, y IPv6-only al final.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Pregunta 5 (8 puntos) — IPv4 vs. IPv6, ND vs. ARP

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010.
>
> **a)** Presente los stacks de referencia de ambos protocolos. Ejemplifique al menos dos protocolos con los que ambos coexisten de capa superior y al menos dos protocolos de capa inferior. **b)** Algunos protocolos definidos dentro de ICMP debieron ser re-definidos en ICMPv6. Mencione el reemplazo del protocolo ARP y describa no menos de 4 cambios significativos en su implementación en Ipv6.

Parte a) — Stacks de referencia

Resuelto en **§2.4**: los dos stacks completos, los dos protocolos de capa superior (**TCP** y **UDP** son los más claros; **DNS** con AAAA también), los dos de capa inferior (**Ethernet** y **PPP**), y la mención de que coexisten en la misma red física, distinguidos por el Ethertype (`0x0800` / `0x86DD`).

Puntos clave: los dos stacks, los 2 protocolos de capa superior, los 2 de capa inferior, y que **la capa 2 no cambia**. Con eso están los 4 puntos.

Parte b) — El reemplazo de ARP y los 4 cambios

Resuelto en **§2.3**; transcribo la respuesta lista para el examen:

El reemplazo: Neighbor Discovery (ND), un **conjunto de 5 mensajes de ICMPv6**: RS (133), RA (134), NS (135), NA (136) y Redirect (137).

Los cambios (los 5 de la solución oficial):

1. **ND es parte de ICMPv6, en su totalidad, y se transporta sobre IPv6.** No es un protocolo independiente como ARP: vive dentro de ICMPv6.
2. **Se utiliza multicast para Neighbor Solicitation en vez de broadcast.** El NS va a `ff02::1:ffxx:xxxx` (link-local), no a `FF:FF:FF:FF:FF:FF`.
3. **Los mapeos son de direcciones link-layer a IPv6 en vez de link-layer a IPv4.**
4. **ND combina la resolución de direcciones con la autoconfiguración y la detección de duplicados.** ARP solo resuelve; ND además configura el host (SLAAC) y detecta colisiones (DAD).
5. **Los nodos pueden usar mensajes NS para propagar cambios en su configuración.**

Justificaciones que suman puntos (no están en la lista oficial, pero demuestran comprensión):

- **Del cambio 2:** los routers **no reenvían** ese multicast, así que el scope **no sale del enlace**. Con ARP (broadcast) un router con *proxy ARP* responde y el broadcast inunda. El multicast es **más eficiente y más seguro**: solo los interesados responden.
- **Del cambio 4:** ND habilita el **IPv6 zero-touch**: el host se enchufa y se configura solo, sin DHCP.

- **Sobre el cambio 5:** con ARP, si un host cambia su IP (failover, roaming), los demás quedan con la caché vieja hasta que expira. Con ND, manda un **NA no solicitado** para actualizarla proactivamente.

> **Nota sobre la solución oficial:** la parte a) de la solución oficial está **vacía** en el PDF (solo viene la b). Si en el examen te piden a) y b), resolvé a) por tu cuenta con los stacks de referencia, que es estándar de la materia.

3.2 Examen 2009-12, Pregunta 5 (6 puntos) — DAD y NDP

> **Fuente:** Examen `sirc0912` — diciembre 2009. **Enunciado:** Considere ICMPv6, familia de protocolos de control en IPv6. El mismo implementa diversos protocolos… a) [mencione dos protocolos que usan NS/NA]. b) DAD no está implementado en IPv4. NDP es la versión IPv6 de ARP, que en vez de ser un protocolo independiente de IPv(v4), también utiliza mensajes IPv6 para su implementación.

Respuesta oficial:

> **a) DAD** – Duplicated Address Detection protocol y **NDP** – Neighbor Discovery Protocol. **b)** DAD no está implementado en IPv4. NDP es la versión IPv6 de ARP, que en vez de ser un protocolo independiente de IP(v4), también utiliza mensajes IPv6 para su implementación.

Ampliación del apartado a): los dos protocolos de ICMPv6 que usan NS/NA.

1. **DAD (Duplicate Address Detection).** Usa **Neighbor Solicitation (135)** para preguntar si una dirección ya está en uso, y espera un **Neighbor Advertisement (136)**. Es la única función que consulta *proactivamente* si su **propia** dirección está libre.
2. **NDP (Neighbor Discovery).** Usa **NS/NA** para el mapeo MAC ↔ IPv6 (resolución), y además para la autoconfiguración y la detección de duplicados.

Comparación con IPv4:

Aspecto	IPv4	IPv6
ARP	Protocolo independiente (`RFC 826`)	**Parte de ICMPv6** (`RFC 4443`)
DAD	**No existe**	**Sí**, todo host lo hace antes de usar una dirección
Solicitud	**Broadcast**	**Multicast** (solicited-node)
Proxy ARP	**Sí**	**No**
Autoconfiguración	No (DHCP o manual)	**Sí** (SLAAC)

¿Por qué DAD no existe en IPv4? Las tres razones de §1.9: (1) en IPv4 la dirección se configura manualmente o por DHCP, así que la colisión es un error humano raro, mientras que con **SLAAC** es estructuralmente frecuente; (2) dos NICs con la misma MAC generan **exactamente la misma dirección**, y DAD lo detecta; (3) **DAD necesita multicast** (NS/NA), y en IPv4 el ARP request es un **broadcast** que no sirve así.

3.3 Examen 2010-07, Pregunta 5 (8 puntos) — modelos, tipos y transición

> **Fuente:** Examen `sirc1007` — julio 2010.
>
> **a)** En el curso se presentaron los tres modelos de comunicación soportados por IPv6, con sus correspondientes tres tipos de direcciones. Mencione cada uno de los tres tipos y descríbalos en una o dos frases a cada uno. **b)** Los esquemas de transición pueden agruparse en tres tipos principales. Descríbalos brevemente y da un ejemplo de cada uno. **c)** Suponga que se transmite un segmento UDP de 1000 bytes de largo sobre IPv4 y sobre IPv6. ¿En qué caso se transmiten más bytes? Fundamente.

Respuesta oficial: a) y b) remiten a las transparencias; **c)** "Sin opciones, un cabezal IPv6 tiene 40 bytes de largo y el cabezal IPv4 tiene 20 bytes de largo. A mismo payload, IPv6 tiene mayor overhead. Por consiguiente, se transmitirán más bytes con IPv6."

Parte a) — Los 3 modelos y los 3 tipos

Resueltos en **§1.5** y **§1.6**. Resumen para el examen:

Modelo	Tipo de dirección	En una frase
Unicast	**Unicast global** (`2000::/3`)	Un emisor, un receptor; cada dirección identifica un nodo. El 99 % del tráfico
Multicast	**Multicast** (`ff00::/8`)	Un emisor, muchos receptores; el prefijo `ff` limita la difusión según el ámbito (`ff02::/16` = enlace)
Anycast	*(no es un tipo de dirección)*	Muchos emisores, muchos receptores; **varios nodos con la MISMA dirección unicast**, y el router encamina al más cercano

Punto clave: aunque el enunciado dice "tres modelos con sus correspondientes tres tipos de direcciones", **el anycast no tiene tipo de dirección propio**: usa una dirección unicast normal. Si hay que forzar la correspondencia, el "tipo" del anycast es **unicast**, y lo especial es que **varios nodos la comparten**.

Parte b) — Los 3 esquemas de transición

Resueltos en **§1.10**: **dual-stack** (ej: servidor con AAAA + A), **tunnelling** (ej: Teredo, 6to4, ISATAP) y **traducción** (ej: NAT64/DNS64, `RFC 6146` / `6147`). Para el apartado basta una frase por esquema y un ejemplo.

Parte c) — El cálculo del overhead

```
Segmento UDP de 1000 bytes de payload.

Sobre IPv4:
   cabecera IP  = 20 bytes (sin opciones)
   segmento UDP = 8 (cabecera UDP) + 1000 (payload) = 1008
   TOTAL        = 20 + 1008 = 1028 bytes

Sobre IPv6:
   cabecera IP  = 40 bytes (fija)
   segmento UDP = 8 + 1000 = 1008
   TOTAL        = 40 + 1008 = 1048 bytes

⇐ Se transmiten MÁS BYTES con IPv6: 1048 > 1028
Diferencia exacta = 40 − 20 = 20 bytes = 20/1028 = 1,9 % de overhead extra.
```

Interpretación. Es el **costo perenne** de la transición: 20 bytes más de cabecera por datagrama. En conexiones con muchos paquetes chicos eso **consume ancho de banda**. Las mitigaciones: las cabeceras de extensión agregan overhead por otro lado, y las **jumbograms** (`Payload Length = 0` con la opción Jumbo Payload) permiten datagramas tan grandes que el overhead relativo es despreciable.

3.4 Examen 2007-03, Pregunta 3 (10 puntos) — RARP

> **Fuente:** Examen `sirc0703` — marzo 2007. **Enunciado:** Describa en que escenario se utiliza el protocolo RARP y su funcionamiento.

Respuesta oficial:

> ARP resuelve el problema de encontrar **qué** dirección Ethernet corresponde a una dirección IP dada. A veces se debe resolver el problema **reverso**: dada una dirección Ethernet, ¿cuál es la dirección IP que le corresponde?
>
> Esto ocurre por ejemplo en el escenario en que hay **estaciones de trabajo tontas**, sin disco, que deben ser reiniciadas. Dicha máquina normalmente obtiene la imagen del sistema operativo de un archivo remoto, pero **antes debe tener una dirección IP**.
>
> La primera solución que se encontró fue usar **RARP**. Este protocolo permitía a las máquinas reiniciadas hacer un **broadcast de Ethernet** enviando una consulta de IP con su dirección Ethernet. Un servidor de RARP recibe la solicitud, realiza una búsqueda en sus archivos de configuración y envía una respuesta con la dirección IP.
>
> Usar RARP es **mejor que grabar la IP en la imagen del sistema operativo** ya que permite **usar la misma imagen para todos los equipos**.
>
> La **desventaja** de RARP es que los **broadcast Ethernet no son reenviados por los routers**, por lo que se necesita un **servidor RARP en cada subred**. Por ello se diseñaron otros protocolos como **BOOTP y DHCP**.

Análisis y ampliación para los 10 puntos.

Los tres elementos que se piden están en la oficial: el **escenario** (workstation tonta sin disco, boot remoto), el **funcionamiento** (broadcast de Ethernet con la MAC → el servidor busca en su tabla → responde con la IP) y la **comparación con ARP** (ARP va de IP → MAC; **RARP va de MAC → IP**, el problema inverso).

Lo que agrego:

(a) El escenario exacto. Un equipo sin disco no tiene dónde guardar su configuración de red. Necesita (1) una IP para hablar con la red, y (2) después, la imagen del OS por TFTP. **RARP resolvía el punto (1)**, y TFTP el (2).

(b) Por qué es mejor que la IP fija en la imagen. Es una razón de **mantenimiento**:

```
Con IP fija en la imagen:
  Cada equipo tiene una imagen DISTINTA (con su IP adentro).
  500 PCs = 500 imágenes que mantener.
  Cambiar el plan de numeración = rehacer 500 imágenes.

Con RARP:
  Todos usan LA MISMA imagen genérica.
  El plan de numeración vive SOLO en el servidor RARP.
  ⇐ Un solo lugar que actualizar.
```

(c) Las limitaciones que prepping su sucesión:

1. **No se reenvía por los routers.** Un broadcast de Ethernet **se queda en el enlace** ⇒ **un servidor RARP por subred**. Con muchos enlaces, son muchos servidores.
2. **RARP solo devuelve la IP.** No devuelve máscara de subred, ni gateway, ni DNS. Para eso están BOOTP y DHCP.
3. ⚠️ **Tema en desuso.** RARP es de **1984** y **hoy está completamente reemplazado por DHCP**, que hace todo lo que RARP hacía y mucho más. No se despliega. Se pregunta por historia y por el **concepto** de "resolver el problema inverso", que reaparece en el **DHCP Discover**: el cliente (que solo tiene su MAC) pregunta "soy esta MAC, ¿qué configuración me corresponde?", y el servidor responde con IP, máscara, gateway, DNS y la **duración** del lease.

La línea histórica que hay que llevarse:

```
RARP (1984)     →  resuelve solo la IP
    ↓
BOOTP (1984)    →  IP + más parámetros, pero con la IP fija en el archivo
    ↓
DHCP (1993)     →  IP + máscara + gateway + DNS + duración del lease + más
    ↓
SLAAC (IPv6)    →  el host se autoconfigura SOLO, leyendo los RA de los
                   routers, y DAD verifica que no haya colisión
```

Dato clave: el **DHCP Discover es, en esencia, un RARP mejorado**.

PARTE 4 — CIERRE

📌 Resumen

1. **El agotamiento de IPv4 es un hecho**: RIPE cerró su última asignación /22 el **25-nov-2019**. El **28-mar-2026** IPv6 superó el **50,10 %** de acceso a Google, pero **la cifra depende de qué se mide** y es desigual (China < 5 %).
2. La **cabecera IPv6 son 40 bytes fijos**. **Desaparecen** `Header Checksum`, `IHL`, `Fragment Offset` y `Options`. **Aparece** el **Flow Label** de 20 bits.
3. El **Flow Label** (`RFC 6437`) identifica un flujo **sin la 5-tupla**, útil con tráfico **fragmentado** o **cifrado**. Retrocompatible: si nadie lo usa, se comporta como IPv4.
4. Las **cabeceras de extensión** se encadenan por `Next Header`, y el **`Payload Length` las EXCLUYE** (bug de `RFC 2460`, corregido por `RFC 2719`).
5. Los **3 modelos**: **unicast** (1→1), **multicast** (`ff00::/8`, 1→muchos, con ámbito), **anycast** (muchos→muchos). **Anycast NO es un tipo de dirección**: es un servicio sobre unicast.
6. Los **tipos de dirección**: unicast global `2000::/3`, **link-local** **`fe80::/10`** (se genera sola), multicast `ff00::/8`, **unique local** **`c00::/7`.** Site-local está deprecado.**
7. El **`/64` es la unidad de subred**, porque SLAAC necesita 64 bits de IID (EUI-64, o aleatorio con Privacy Extensions, que es el default actual).
8. **ICMPv6 es esencial**: Packet Too Big (2), Destination Unreachable (1), Time Exceeded (3), Parameter Problem (4), Echo (128/129), ND (133-137).
9. **ND vs. ARP**: ND es parte de ICMPv6, usa **multicast**, mapea ↔IPv6, **combina 3 funciones** y **propaga cambios**. Son 5 mensajes (RS, RA, NS, NA, Redirect).
10. **DAD** detecta duplicados con NS + temporizador de 1 s. No existe en IPv4 porque las direcciones se configuran manualmente. Los **3 esquemas de transición** son **dual-stack**, **tunnelling** (Teredo, 6to4, ISATAP) y **traducción** (NAT64/DNS64, 464XLAT).

✅ Autoevaluación

1. ¿Por qué desapareció el `Header Checksum` de la cabecera IPv6? ¿No es un problema no verificarlo?

▶ Respuesta

2. ¿Qué es el `Flow Label` y para qué sirve, si ya tenemos la 5-tupla?

▶ Respuesta

3. ¿Por qué anycast no es un tipo de dirección?

▶ Respuesta

4. ¿Por qué existe DAD en IPv6 y no en IPv4?

▶ Respuesta

5. ¿Qué diferencia hay entre `Packet Too Big` (ICMPv6 tipo 2) y `Fragmentation Needed and DF set` (ICMPv4 tipo 3 código 4)?

▶ Respuesta

6. ¿Por qué el `Payload Length` de IPv6 excluye las cabeceras de extensión? ¿Qué problema resuelve?

▶ Respuesta

7. ¿Por qué un router IPv6 nunca fragmenta? ¿Qué consecuencias tiene?

▶ Respuesta

8. En el esquema **dual-stack**, ¿qué ventaja y qué desventaja tiene para un operador?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1012` / `sirc1012`** — P5 (8 pts: stacks de referencia, coexistencia, **ND vs. ARP con ≥4 cambios**)
- **Examen `sirc0912`** — P5 (6 pts: DAD y NDP, comparación con IPv4)
- **Examen `sirc1007`** — P5 (8 pts: 3 modelos de comunicación, 3 tipos de dirección, 3 esquemas de transición, overhead IPv4 vs. IPv6)
- **Examen `sirc0703`** — P3 (10 pts: RARP, escenario y funcionamiento)
- **Examen `sirc0907`** — P5 (ventajas y desventajas de IPv6)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 (IPv6)
- **RFC 8200** — Internet Protocol, Version 6 (2017, *Internet Standard*; **obsoleta RFC 2460**; §5 fija el **MTU mínimo de 1280**)
- **RFC 4443** — ICMPv6 for IPv6 (tipos 1-4 y 128-137)
- **RFC 6437** — Flow Label (2012)
- **RFC 2719** — Corrección del bug del `Payload Length` de `RFC 2460`
- **RFC 6052** — Prefijo bien conocido `64:ff9b::/96` para NAT64 (2010)
- **RFC 6146** — NAT64 (2011) · **RFC 6147** — DNS64 (2011)
- **RFC 7915** — SIIT (2016, **obsoleta RFC 6145**) · **RFC 6333** — DS-Lite (2011)
- `draft-ietf-v6ops-464xlat` — **464XLAT sigue en borrador**
- **RFC 5722** — Fragmentos solapados en IPv6 (prohibidos) · **RFC 6946** — Fragmentos atómicos · **RFC 5095** — depreca Routing Header tipo 0
- **RFC 8305** — Happy Eyeballs (2017, *obsoleta RFC 6555*)
- **RFC 9926** — NDP: registro de prefijos entre routers vecinos (feb-2026)

- **Clase siguiente:** 37 — ICMP, ping y traceroute + Repaso y Simulacro 4